Robust Crease Detection and Curvature Estimation of Piecewise Smooth Surfaces from Triangle Mesh Approximations Using Normal Voting

D. L. Page, A. Koschan, Y. Sun, J. Paik, and M. A. Abidi
Imaging, Robotics, and Intelligent Systems Laboratory
The University of Tennessee
Knoxville, TN 37996
page@iristown.engr.utk.edu

Abstract

In this paper, we describe a robust method for the estimation of curvature on a triangle mesh, where this mesh is a discrete approximation of a piecewise smooth surface. The proposed method avoids the computationally expensive process of surface fitting and instead employs normal voting *to achieve robust results. This method detects crease discontinuities on the surface to improve estimates near those creases. Using a voting scheme, the algorithm estimates both principal curvatures and principal directions for smooth patches. The entire process requires one user parameter—the voting neighborhood size, which is a function of sampling density, feature size, and measurement noise. We present results for both synthetic and real data and compare these results to an existing algorithm developed by Taubin.*

1. Introduction

The estimation of *surface curvature* plays a key role in many computer vision algorithms such as scene segmentation and object recognition [10]. The challenge is that curvature is often difficult to compute with just a discrete approximation and not the original smooth surface itself. Additionally, we typically assume a scene contains rigid objects, and therefore the original surfaces are not just smooth but piecewise smooth [2]. In this paper, we address the problem of estimating curvature of piecewise smooth surfaces from triangle mesh approximations, despite crease discontinuities and possible surface noise.

What do we mean by surface curvature? If we follow Taubin [10], we see that for a point p on a smooth surface S, we can completely specify the surface curvature at p with two scalars, κ_p^1 and κ_p^2, and two orthogonal unit vectors, T_1 and T_2. The scalars are the *principal curvatures* and correspond to the maximum and minimum *normal curvatures* at p. Associated with the curvatures are the *principal directions*, which are the vectors T_1 and T_2. We seek an algorithm that estimates both the principal curvatures and principal directions for each vertex of a mesh.

In the literature two methods show the most promise: Taubin [10] and Tang and Medioni [9]. These methods differ from the traditional range image approaches [2]. We suggest that both methods are *curve fitting* algorithms that fit a family of curves around a point and use this ensemble of curves to estimate curvature. Other curve fitting methods include [1, 6]. By contrast, *surface fitting* methods [3, 8] locally fit an analytic surface around a point and compute curvature from the derivatives of this function. Other approaches [5, 11] use the topology and geometry of the mesh directly to estimate *total curvature* for a region of the mesh.

We propose a novel curve fitting method with the following traits:

- detects piecewise smooth creases,
- smoothes measurement error, and
- estimates principal curvatures *and* principal directions.

In the following sections, we begin with a brief background of Taubin's discrete formulation of curvature estimation. We then present the details of our algorithm and follow with experimental results. These results use both synthetic and real data to compare our algorithm to Taubin's method. Finally, we conclude with a few closing remarks.

2. Discrete estimation

Taubin [10] shows that the symmetric matrix

$$M_p = \frac{1}{2\pi} \int_{-\pi}^{\pi} \kappa_p(T_\theta) T_\theta T_\theta^t d\theta \,, \qquad (1)$$

0-7695-1272-0/01 $10.00 © 2001 IEEE

with superscript t denoting transpose, has eigenvectors that are equivalent to the principal directions $\{T_1, T_2\}$ and eigenvalues that are related by a fixed homogeneous linear transformation to the principal curvatures as

$$\begin{aligned} \kappa_p^1 &= 3m_p^1 - m_p^2 \\ \kappa_p^2 &= 3m_p^2 - m_p^1 \end{aligned} \tag{2}$$

where m_p^1 and m_p^2 are the eigenvalues of M_p associated with T_1 and T_2, respectively. The other eigenvalue is zero and corresponds to the eigenvector equal to N at p. Taubin approximates equation (1) as

$$\tilde{M}_v = \frac{1}{2\pi} \sum w_i \kappa_i T_i T_i^t \tag{3}$$

for a finite set of directions T_i where $\tilde{M}_v$ denotes approximation of M_p at vertex v. The weight w_i is a discrete version of the integration step and has the constraint $\sum w_i = 2\pi$. Taubin's algorithm computes $\tilde{M}_v$ and then finds the eigenvectors and eigenvalues that lead to the principal directions and principal curvatures with equation (2).

The question that we now face is how do we estimate κ_i and T_i in equation (3) from a discrete triangle mesh. Taubin employs a truncated Laurent series, which works well if the mesh has little or no surface noise. Tang and Medioni [9], however, have developed their approach with noise in mind. They formulate a matrix similar to equation (3) but do not relate the eigenvalues to the principal curvatures. Our algorithm extends the robust capabilities of Tang and Medioni with the more complete formulations from Taubin. We call our method *normal voting*.

3. Normal voting

The basic idea of normal voting is to select a geodesic neighborhood of a vertex where the triangles in this neighborhood first *vote* to estimate the normal at the vertex. Then, with these normal estimates, the vertices in the same neighborhood vote to estimate curvature.

3.1. Vote collection

The first step in normal voting is to find the triangles that are close, in a geodesic sense, to the vertex of interest. Our *geodesic neighborhood* problem is to find the m triangles that are the closest geodesic neighbors of a given vertex v. Kimmel and Sethian [4] present an algorithm that solves this problem in $O(m \log m)$ time. We denote the collection of triangles that are geodesic neighbors of v as surface patch S_v^0. Figures 3(a) and 3(b) show examples of two different neighborhood sizes.

The next step involves the voting of the triangle faces $f_i \in S_v^0$ at the vertex v. We first consider how v collects votes where we assume that each f_i generates a normal vote N_i at v. Medioni et al. [7] suggest representing N_i as a covariance matrix $V_i = N_i N_i^t$ and collecting votes as a weighted matrix sum

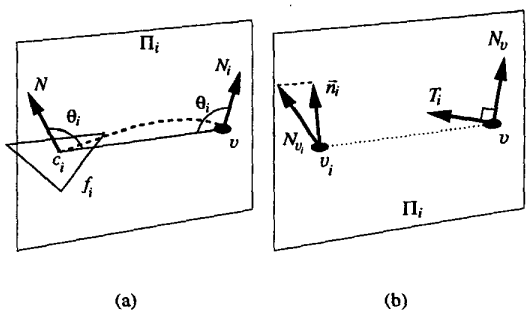

Figure 1. These line drawings show the normal voting geometries. (a) A triangle votes at a vertex. We rotate N by $(2\theta_i - \pi)$ in plane Π_i. (b) A vertex votes for curvature at another vertex. The vectors N_v, $\vec{n}_i$, and T_i lie in the plane Π_i and $\vec{n}_i$ is the projection of N_{v_i}.

$$V_v = \sum w_i V_i = \sum w_i N_i N_i^t \tag{4}$$

where the summation is over the m neighborhood of v. Unfortunately, the downside is that we lose absolute sign orientation ($N_j = -N_i$, $N_i N_i^t = N_j N_j^t$). We can solve this problem with an *ad hoc* solution that we discuss later.

The next question is how does f_i generate N_i at v. With some insight, we see that an interesting approach is to fit a smooth curve from f_i to v and allow the normal to follow this curve. What type of curve should we use? Medioni et al. [7] argue that the most appropriate one is a circular arc with the shortest possible arc length. Following this argument, we construct the geometry in Figure 1(a). The curve in this figure lies entirely in the plane Π_i, which contains the face normal N—rooted at centroid c_i—and the point v. We can compute θ_i in the figure as

$$\cos\theta_i = \frac{N^t \vec{vc}_i}{\|\vec{vc}_i\|} \tag{5}$$

where $\vec{vc}_i = c_i - v$. This equation leads to the normal estimate

$$N_i = N - 2\cos\theta_i \frac{\vec{vc}_i}{\|\vec{vc}_i\|} . \tag{6}$$

The above formulation is computationally more efficient than the voting fields and matrix rotations of Medioni et al.

If we recall equation (4), we now only lack the weighting term w_i. Two factors effect this term: surface area of f_i and geodesic distance g_i of c_i from v. We choose an exponential

decay to reflect these factors

$$w_i = \frac{A_i}{A_{max}} \exp\left(-\frac{g_i}{\sigma}\right) \tag{7}$$

where σ controls the rate of decay, A_i is the area of f_i, and A_{max} is the area of the largest triangle in the entire mesh. If g_m is the maximum geodesic distance that bounds the m geodesic neighborhood, then we typically set $g_m = 3\sigma$ since votes beyond have negligible influence. We note that m, and by extension σ, is the only user specified parameter of the normal voting algorithm.

3.2. Crease detection

The third step is to decompose V_v in equation (4) with eigen analysis and to classify v. Since V_v is a symmetric semi-definite matrix, eigen decomposition generates real eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$ with corresponding eigenvectors $\vec{e}_1$, $\vec{e}_2$, and $\vec{e}_3$. Medioni et al. [7] define the following relationships

$$\begin{array}{ll} \mathcal{S}_s = \lambda_1 - \lambda_2 \,, & \text{surface patch saliency;} \\ \mathcal{S}_c = \lambda_2 - \lambda_3 \,, & \text{crease junction saliency; and} \\ \mathcal{S}_n = \lambda_3 \,, & \text{no preferred orientation saliency.} \end{array} \tag{8}$$

We suggest that the maximum of the above relationships determines how we classify v

$$\max\{\mathcal{S}_s, \varepsilon\mathcal{S}_c, \varepsilon\mathcal{S}_n\} = \begin{cases} \mathcal{S}_s : & \text{surface } N_v = \vec{e}_1 \\ \varepsilon\mathcal{S}_c : & \text{crease } T_v = \vec{e}_3 \\ \varepsilon\mathcal{S}_n : & \text{no orientation} \end{cases} \tag{9}$$

where $0 \leq \varepsilon < \infty$ is a system constant that controls the relative significance of the saliencies.

We demonstrate the impact of ε with examples. First, consider the case where $\varepsilon \to 0$. This system always classifies a vertex as a surface patch. Conversely, the system where $\varepsilon \to \infty$ will never declare a vertex as a surface patch. These extremes reveal that ε governs the decision point for crease detection. When designing a system, we fix ε to discriminate the types of creases that we expect while allowing a certain level of noise. For most applications, the system $\varepsilon = 2$ offers a balanced compromise. As a rule of thumb, we can use $\phi = 2 \arctan\sqrt{\frac{1}{\varepsilon+1}}$ to design for a specific crease angle.

3.3. Curvature estimation

With our normal estimate N_v for smooth patches from the previous sections, we can generate a finite set of curvature votes κ_i in the directions T_i around v. We can plug these values into equation (3) and estimate the principal curvatures κ_v^1 and κ_v^2 and directions T_1 and T_2 at v. Taubin uses only the vertices that share an edge with v. We enlarge this neighborhood with our geodesic algorithm to find the vertices within $g_m = 3\sigma$ boundary. For the weight term, we again use a decay function as in equation (7) except that we exclude A_i and constrain with the sum $\sum w_i = 2\pi$. We must now address how to estimate κ_i and T_i for each vertex v_i in the m neighborhood.

The geometry in Figure 1(b) illustrates the direction for T_i with

$$T_i = \frac{\vec{t}_i}{\|\vec{t}_i\|} \,, \quad \vec{t}_i = \vec{vv}_i - (N_v^t \vec{vv}_i) N_v \tag{10}$$

where $\vec{vv}_i = v_i - v$. For the normal curvature κ_i in the direction of T_i, we use a discrete definition for curvature with the change in turning angle ϑ_i and arc length s

$$\kappa_i = \frac{\Delta\vartheta_i}{\Delta s} \,. \tag{11}$$

For the normal curvature, we must project N_{v_i} into the plane Π_i that contains N_v—rooted at v—and v_i as

$$\vec{n}_i = N_{v_i} - (P_i^t N_{v_i}) P_i \tag{12}$$

where $\vec{n}_i$ is the projection and $P_i = N_v \times T_i$. The change in turning angle is

$$\cos(\Delta\vartheta_i) = \frac{N_v^t \vec{n}_i}{\|\vec{n}_i\|} \,. \tag{13}$$

Finally, we estimate the change in arc length $\Delta s = g_i$ as the geodesic distance from v_i to v using the output from Kimmel and Sethian's algorithm [4]. We equate the sign of κ_i to be the same as the sign of $T_i^t \vec{n}_i$.

We have reached our goal. We discuss a couple of notes, however. First, we only compute surface curvature if we classify a vertex as a surface patch. We do nothing for vertices that we declare to be creases or to have no preferred orientation. Additionally, as we gather the vertices with the neighborhood algorithm, we do not cross crease vertices. This restriction constrains S_v^0 to the same smooth patch as v and thus improves the curvature estimate.

4. Experimental Results

We now investigate the experimental results for the algorithm we have presented. We begin with the qualitative results in Figures 2 and 3. Theses figures illustrate the performance of normal voting for a fandisk model. To this model, we have added noise to each coordinate of the vertices where we specify the noise as the variance of a Gaussian distribution. We show two variance values of 0.05% and 0.1% of the average edge length l_{ave} in Figures 2(a) and 2(b), respectively. In Figures 2(c) and 2(d), we place

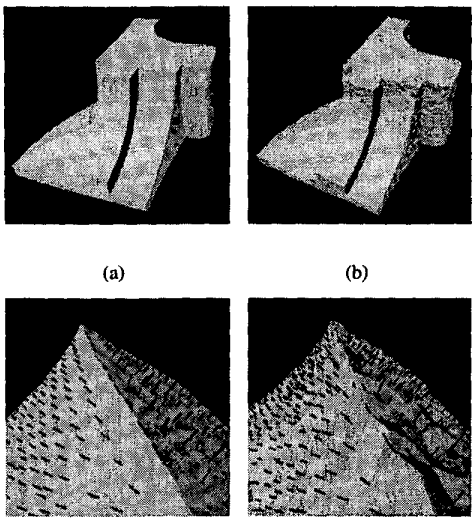

Figure 2. Crease detection for fandisk with (a,c) 0.05% and (b,d) 0.1% Gaussian noise. The zoom views (c,d) are the same crease from (a,b). The model is available in the data distribution at http://research.microsoft.com/ hoppe.

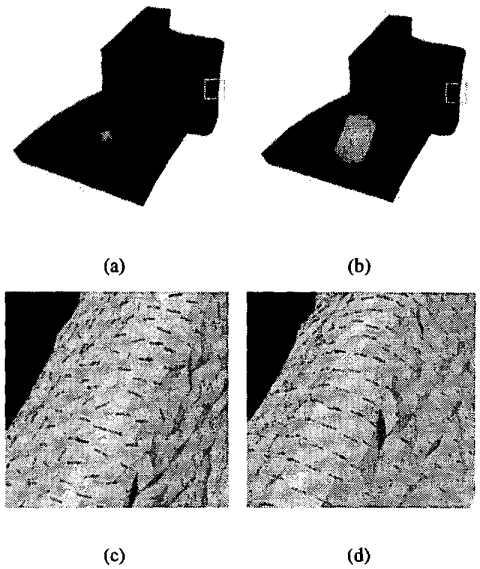

Figure 3. Effect of neighborhood size with (a,c) $\sigma = l_{ave}$ and (b,d) $\sigma = 5l_{ave}$. Vectors in (c,d) show direction of maximum curvature for the boxed regions in (a,b).

a black (blue for color illustrations) normal at each smooth vertex and a black (purple) tangent at each crease vertex. The zoom views demonstrate the robust detection of the creases despite severe reflex angles of neighboring triangles and the robust estimation of normals even near creases. If we vary the neighborhood size, we see the effects in Figures 3(c) and 3(d). These zoom views have black (red) vectors for the maximum principal directions. The surfaces have 0.1% Gaussian noise and each view shows a different neighborhood size with $\sigma = l_{ave}$ and $\sigma = 5l_{ave}$, respectively. The zoom area is the light grey (yellow) box in Figures 3(a) and 3(b), and these figures also show an example of each neighborhood as grey (yellow and red) target splats. The $\sigma = l_{ave}$ neighborhood in Figure 3(a) is equivalent to the umbrella neighborhood of Taubin. As these figures show, the enlargement of the neighborhood allows significant improvement in the local agreement of principal directions.

Figures 4(g)–4(o) provide a more quantitative analysis of the performance. In these figures, we graph the error of the algorithm for both synthetic and real data where we use ground truth to establish the error. For these graphs, we manipulate three variables: surface type, noise level, and neighborhood size. Figures 4(a)–4(c) illustrate the surface types and noise levels for the synthetic data. These meshes are planar, cylindrical, and spherical, respectively. We corrupt the Z coordinate of each vertex with Gaussian noise of either 50%, 10%, 5% or 1% of l_{ave}. From left to right, the figures show the effects of 50%, 10%, and 5% noise. For brevity, we do not show the 1% noise. Figures 4(d)–4(f) illustrate real data generated from an IVP Ranger System, which is a sheet-of-light profile scanner.

We first consider synthetic data to evaluate noise sensitivity. We choose three surface types: planar, cylindrical, and spherical. The graphs in Figures 4(g)–4(i) compare the performance of the normal voting algorithm for two neighborhoods ($\sigma = 3l_{ave}$ and $\sigma = 5l_{ave}$) to Taubin's algorithm. The surface noise for these graphs is 10%. We look at the normal estimation for the plane in Figure 4(g). The error in this graph is as follows $err = (1 - |N_p^t N_v|)$ where $N_p = Z$ is the ground-truth normal and N_v is the estimation. This graph is a histogram plot with vertex bins across the horizontal axis and a log scale for the vertical. Figure 4(h) uses a similar error measure and compares the estimation of the principal directions T_v for the cylinder. We let $T_p = X$ be the ground truth. The third graph in Figure 4(i) compares the estimation of the principal curvatures for the sphere. We use a different error measure $err = |\kappa_p^1 - \kappa_v^1|$ where κ_p^1 is the ground truth and κ_v^1 is the estimate. For each of these graphs, we see a similar trend. The normal voting

algorithm for both neighborhoods provides improved performance over Taubin's algorithm.

Next, we consider the effect of different noise levels. We use the four noise levels to evaluate the algorithms performance with a neighborhood size $\sigma = 3l_{ave}$. Using the previous error measures and graphs, Figures 4(j)–4(l) plot the normal voting error for each surface type and noise level. Although the 50% level seems to overwhelm the normal voting algorithm, the other three levels offer useful results for most applications. Again, these graphs demonstrate the robustness of the algorithm.

Finally, we explore real data from an IVP Ranger System, which is a sheet-of-light profile scanner. The basic output of the scanner is a single range profile in the plane of the sheet of light. For our tests, we stack 512 profiles together to form a 512×512 range image with 256 range bins at 0.62 mm resolution. With a simple algorithm and proper calibration, we convert these range images into appropriate triangle meshes. We again use three surface types as with the synthetic data. With slight modifications, we use the same error measures and graphs as with the synthetic data. Since we do not know the absolute orientation of the objects relative to the scanner, we must account for this uncertainty. For the plane, we average the normal estimates to serve as the ground truth $N_p = \frac{1}{n}\sum N_v$ for each vertex, and for the cylinder, we average the minimum principal direction estimates $T_p = \frac{1}{n}\sum T_v$. These *ad hoc* formulations of ground truth introduce some error, but the error is constant across our analysis and is tolerable. Figure 4(m)–4(o) show the error graphs. These results are similar to the synthetic data where again the normal voting algorithm shows improvement over Taubin's algorithm.

5. Conclusions

We have introduced a new algorithm for estimation of surface normals and principal curvatures on a triangle surface mesh where the mesh is an approximation of a piecewise smooth surface. This algorithm allows the detection of crease discontinuities and is robust to surface noise. The algorithm is non-iterative and has a complexity of $O(nm \log m)$ time where n is the number of vertices in the mesh and m is the neighborhood size as defined by σ. The parameter σ is the only user-specified parameter for the algorithm. We have presented results for this new algorithm using both synthetic and real data where we have demonstrated stable results independent of the surface curvature and noise level. These results highlight the improvements that our extension of Taubin's original algorithm offers.

Acknowledgments

This work is supported by the University Research Program in Robotics under grant DOE-DE-FG02-86NE37968, by the DOD/TACOM/NAC/ARC Program, R01-1344-18, and by FAA/NSSA Program, R01-1344-48/49.

References

[1] X. Chen and F. Schmitt. Intrinsic surface properties from surface triangulation. In *Proceedings of the European Conference on Computer Vision*, pages 739–743, Santa Margherita Ligure, Italy, 1992.

[2] P. J. Flynn and A. K. Jain. On reliable curvature estimation. In *Proceedings of the International Conference on Computer Vision and Pattern Recognition*, pages 110–116, 1989.

[3] H. Hagen, S. Heinz, M. Thesing, and T. Schreiber. Simulation based modeling. *International Journal of Shape Modeling*, 4(3,4):143–164, 1998.

[4] R. Kimmel and J. A. Sethian. Computing geodesic paths on manifolds. In *Proceedings of the National Acadmeny of Sciences*, volume 95, pages 8431–8435, 1998.

[5] C. Lin and M. J. Perry. Shape description using surface triangulation. In *Proceedings of the IEEE Workshop on Computer Vision: Representation and Control*, pages 38–43, 1982.

[6] R. R. Martin. Estimation of principal curvatures from range data. *International Journal of Shape Modeling*, 4(3,4):99–109, 1998.

[7] G. Medioni, M.-S. Lee, and C.-K. Tang. *A Computational Framework for Segmentation and Grouping*. Elsevier, Amsterdam, 2000.

[8] C. Rössl, L. Kobbelt, and S. Hans-Peter. Extraction of feature lines on triangulated surfaces using morphological operators. In *Smart Graphics (AAAI Symposium 2000)*, pages 71–75, Menlo Park, CA, 2000. AAAI Press.

[9] C.-K. Tang and G. Medioni. Robust estimation of curvature information from noisy 3D data for shape description. In *Proceedings of the Seventh International Conference on Computer Vision*, pages 426–433, Kerkyra, Greece, Sept. 1999.

[10] G. Taubin. Estimating the tensor of curvature of a surface from a polyhedral approximation. In *Proceedings of the Fifth International Conference on Computer Vision*, pages 902–907, 1995.

[11] K. Wu and M. D. Levine. 3D part segmentation using simulated electrical charge distributions. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(11):1223–1235, Nov. 1997.

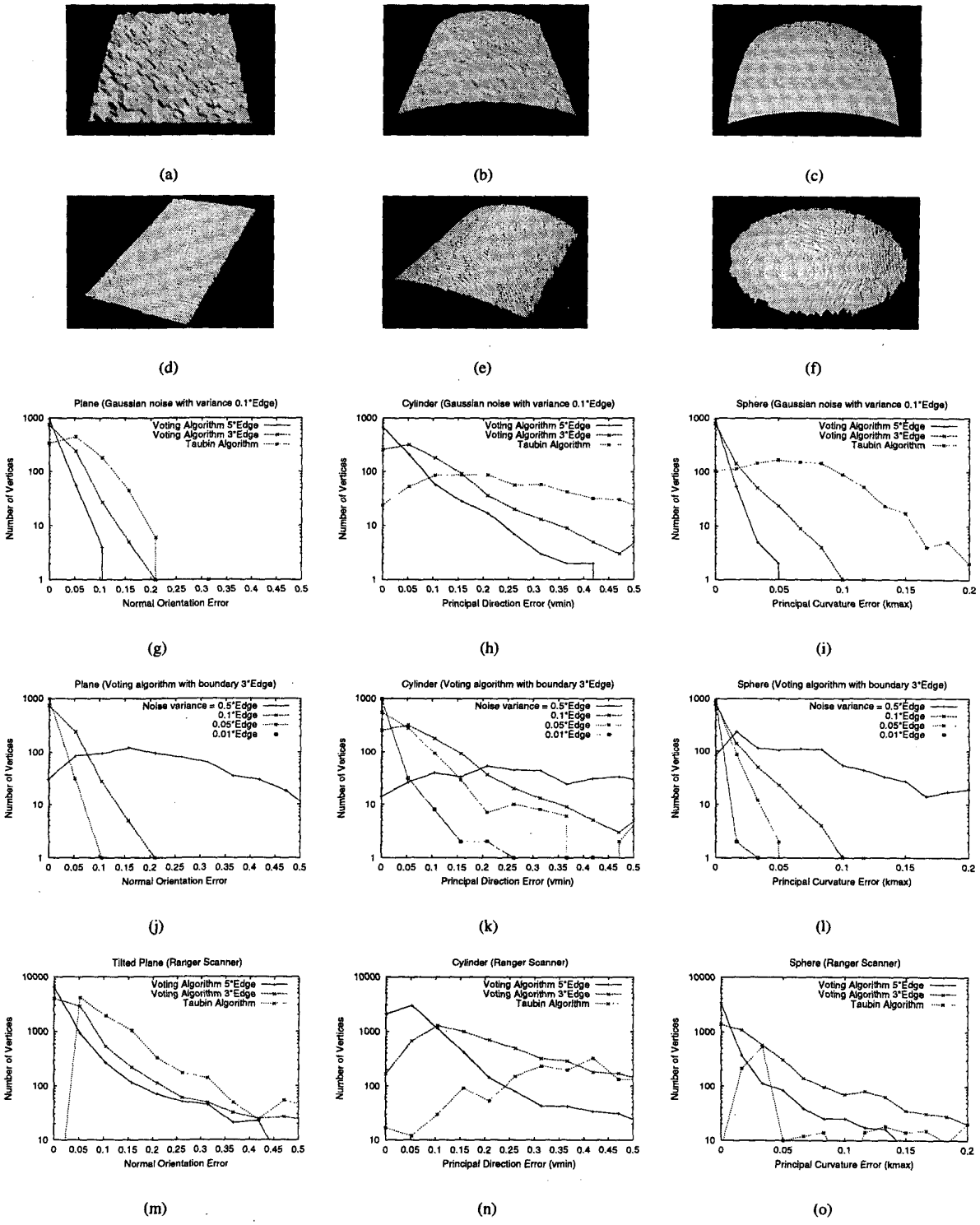

Figure 4. Comparison of (a–c, g–l) synthetic and (d–f,m–o) real data.